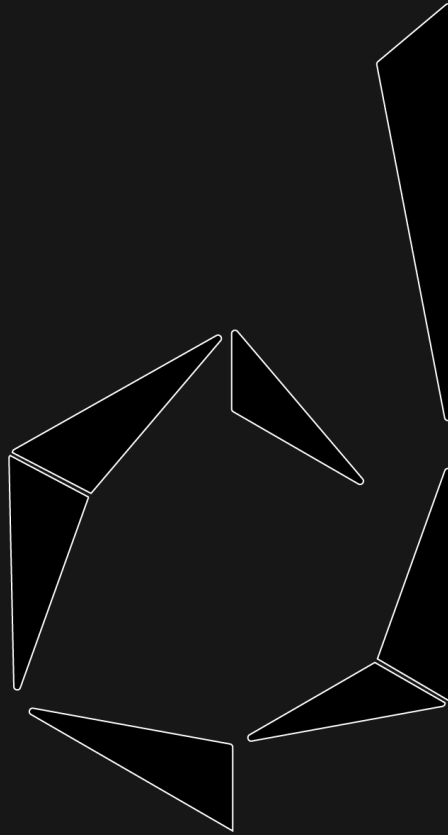


INGENIERÍA MECATRÓNICA



DI_CERO

DIEGO CERVANTES RODRÍGUEZ

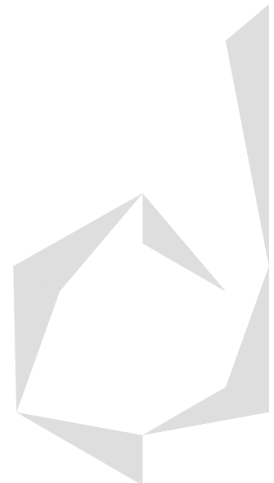
PROGRAMACIÓN: DESARROLLO BACKEND

PYTHON

Introducción a Django

Contenido

Introducción a Django.....	2
Arquitectura MVT (Model View Template)	4
Model: ORM (Object-Relational Mapping) y Migraciones.....	7
Base de Datos SQLite y su Modificación en Settings.py	9
Tipos de Relaciones entre Tablas: Uno a Muchos, Muchos a Muchos y Uno a Uno	11
URLs.py: Archivo de Configuración de Endpoints.....	12
Django Templates: Variables, Filtros y Tags.....	16
Ejemplo: Proyecto Coffee Shop	16
Referencias.....	18



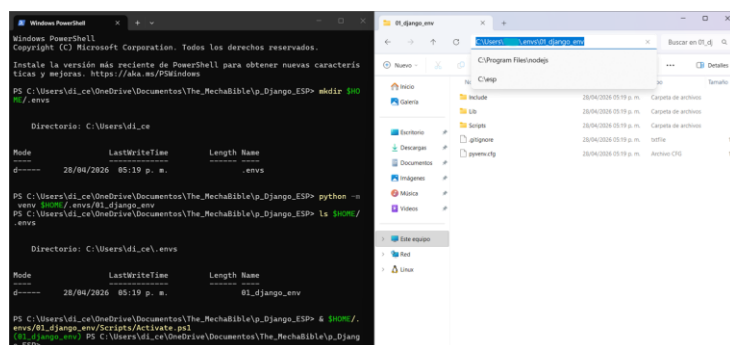
Introducción a Django

Django es un framework orientado a objetos (POO) para desarrollo web escrito en Python, inicialmente fue construido como un generador de blogs, pero poco a poco fue ganando popularidad y ahora es utilizado para construir aplicaciones, más que nada por su facilidad de uso, debido a que nos permite construir apps desde cero a producción en tiempo record. Inicialmente Instagram y Spotify crearon la primera versión de sus aplicaciones en Django.

Django utiliza entornos virtuales, estos se refieren a paquetes que se puedan utilizar en diferentes versiones del sistema operativo de nuestra computadora, sin que se causen colisiones, temas de compilación o versionamiento del código.

Al instalar Django debemos ejecutar el siguiente comando en nuestra terminal CDM o PowerShell, para ello ya debemos haber destinado una carpeta de Python donde se creará el entorno virtual, realizando su instalación y activación de la siguiente forma:

- **mkdir \$HOME/.envs:** En PowerShell a veces conviene crear primero manualmente la carpeta que almacenará los entornos virtuales.
 - El entorno virtual se crea de esta forma si se quiere utilizar **el sistema operativo de Windows en vez de la WSL (Windows Subsystem for Linux)**, que es una opción donde se usa Linux dentro de Windows como el sistema operativo que guarda el entorno virtual.
- **python -m venv \$HOME/.envs/01_django_env:** El comando principal para **crear el entorno virtual** es **python -m venv** y después se debe pasar como parámetro el **path** de donde queremos que se guarde este entorno virtual, en este caso parte de nuestra carpeta home C: al utilizar el comando **\$HOME** y luego indicamos **el nombre de las carpetas posteriores que se crearán para almacenar el entorno virtual**.
 - Una vez que hayamos creado el entorno virtual dentro de la carpeta **~/envs/**, con el comando **ls** podremos ver la carpeta del entorno virtual **01_django_env** creada con ese nombre dentro de ella.
- **& \$HOME/.envs/01_django_env/Scripts/Activate.ps1:** El comando principal para **activar el entorno virtual** es **& path/Scripts/Activate.ps1**, al hacerlo correctamente, el nombre del entorno virtual aparecerá hasta el inicio de nuestra consola CMD o PowerShell entre paréntesis y cuando esto pase, todos los comandos que utilice después se ejecutarán en el entorno virtual, para que después pueda crear una aplicación que sea ejecutable en cualquier versión de mi sistema operativo.
 - Para salir del entorno virtual, debemos ejecutar el comando: **deactivate**



The image shows a PowerShell terminal window and a file explorer window. The terminal window displays the following commands and output:

```
PS C:\Users\ldi_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> mkdir $HOME/.envs

Directorio: C:\Users\ldi_ce

Mode                LastWriteTime         Length Name
----                -
d-----          28/04/2026  05:19 p. m.         .envs

PS C:\Users\ldi_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python -m venv $HOME/.envs/01_django_env
PS C:\Users\ldi_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> ls $HOME/.envs

Directorio: C:\Users\ldi_ce\.envs

Mode                LastWriteTime         Length Name
----                -
d-----          28/04/2026  05:19 p. m.         01_django_env

PS C:\Users\ldi_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> & $HOME/.envs/01_django_env/Scripts/Activate.ps1
PS C:\Users\ldi_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP>
```

The file explorer window shows the contents of the **C:\Program Files\Python310\Scripts** directory, which includes files like **activate.ps1**, **activate.bat**, **python.exe**, **pythonw.exe**, **pip.exe**, **pipw.exe**, **pip3.exe**, **pip3w.exe**, **python3.exe**, and **python3w.exe**.

- Ya que tengamos el entorno virtual activado debemos utilizar el instalador de paquetes de Python **pip** para realizar la instalación de Django dentro de nuestro entorno de desarrollo activado, esto se realizará con el comando: **pip install django**
- **django-admin startproject a_first_project**: Una vez que instalamos Django, para crear nuestro proyecto debemos utilizar el comando **django-admin** y su método **startproject**, luego le asignaremos un **nombre**, para ello debemos evitar usar números al inicio y guiones medios (-) y en vez de eso hay que utilizar guiones bajos (_).
 - **cd a_first_project**: Luego deberemos ingresar a la reciente carpeta creada de nuestro proyecto y ahí deberemos ejecutar un archivo para que podamos correrlo.
 - **python manage.py runserver**: Al ejecutar correctamente los comandos de creación del proyecto Django, se creará el archivo **manage.py** en nuestra carpeta actual, este lo debemos ejecutar y entrar en nuestro localhost dentro del navegador (<http://127.0.0.1:8000/>) para ver si la página principal de Django ha sido lanzada, si esto se ha realizado correctamente, nuestro sitio ha sido configurado y creado.

The screenshot shows a Windows PowerShell terminal window on the left and a file explorer window on the right. The terminal window displays the commands and output for installing Django and creating a new project. The file explorer window shows the contents of the 'a_first_project' directory, including the 'manage.py' file and the 'db.sqlite3' file.

```

PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> python -m venv .\envs\01_django_env
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> .\envs\01_django_env\Scripts\activate.ps1
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> pip install django
Collecting django
  Downloading django-6.0.1-py3-none-any.whl (1.9 MB)
  Collecting asgiref<3.9.1, from django
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.1 kB)
  Collecting sqlparse<0.5.0, from django
  Downloading sqlparse-0.5.0-py3-none-any.whl.metadata (4.7 kB)
  Collecting tzdata (from django)
  Downloading tzdata-2026.2-py3-none-any.whl.metadata (1.4 kB)
  Downloading django-6.0.1-py3-none-any.whl (1.9 MB)
  Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
  Downloading sqlparse-0.5.0-py3-none-any.whl (46 kB)
  Downloading tzdata-2026.2-py3-none-any.whl (299 kB)
  Installing collected packages: tzdata, sqlparse, asgiref, django
  Successfully installed asgiref-3.11.1 django-6.0.1 sqlparse-0.5.0 tzdata-2026.2

[notice] A new release of pip is available: 26.0.1 -> 26.1
[notice] To update, run: python -m pip install --upgrade pip
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> django-admin startproject 01_first_project
CommandError: '01_first_project' is not a valid project name. Please make sure the name is a valid identifier.
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> django-admin startproject a_first_project
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> cd .\a_first_project\
PS C:\Users\ldi_cero\OneDrive\Documents\The_Mechanika\p_django_ESP> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 11 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
April 28, 2026 - 17:19:56
Django version 6.0.1, using settings 'a_first_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-C/CMD-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[28/Apr/2026 17:40:14] "GET / HTTP/1.1" 200 12606
Not found: /favicon.ico
[28/Apr/2026 17:40:14] "GET /favicon.ico HTTP/1.1" 404 2317
  
```

The file explorer window shows the 'a_first_project' directory with the following files and folders:

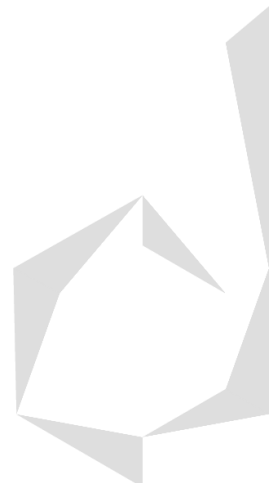
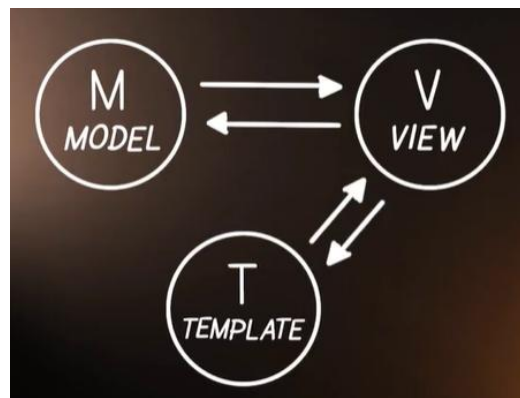
Nombre	Fecha de modificación	Tipo	Tamaño
a_first_project	28/04/2026 05:39 p. m.	Carpeta de archivos	
manage	28/04/2026 05:34 p. m.	Archivo de origen ...	1 KB
db.sqlite3	28/04/2026 05:39 p. m.	Archivo SQLite3	0 KB

- **django-admin --help:** El comando **django-admin** no solamente sirve para crear el proyecto con el comando **startproject**, sino para muchas cosas más, para poder ver todas las cosas que hace debemos ejecutar su comando **--help**.
- **python manage.py --help:** El comando **python manage.py** no solamente sirve para ejecutar el proyecto con el método **runserver**, sino para muchas cosas más, para poder ver todas las cosas que hace debemos ejecutar su comando **--help**.

Arquitectura MVT (Model View Template)

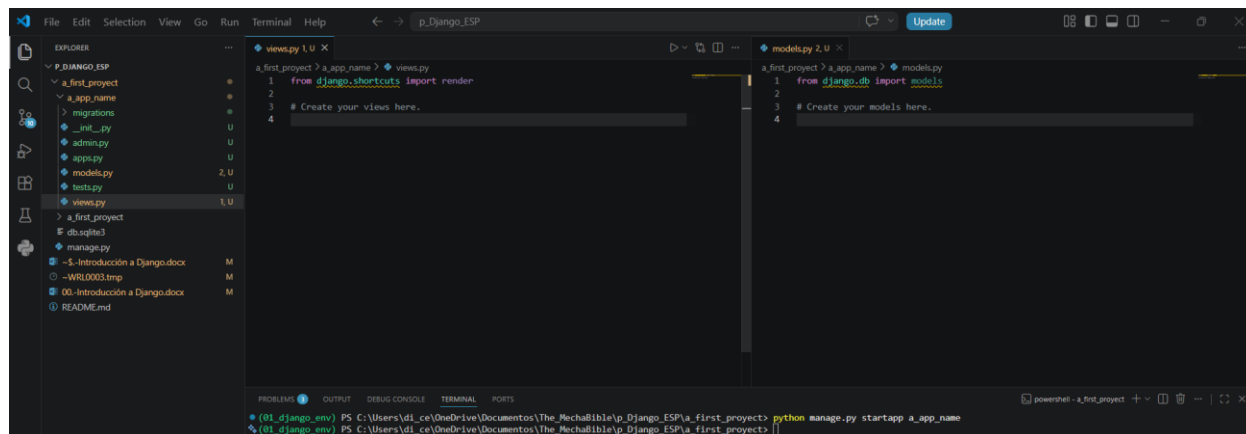
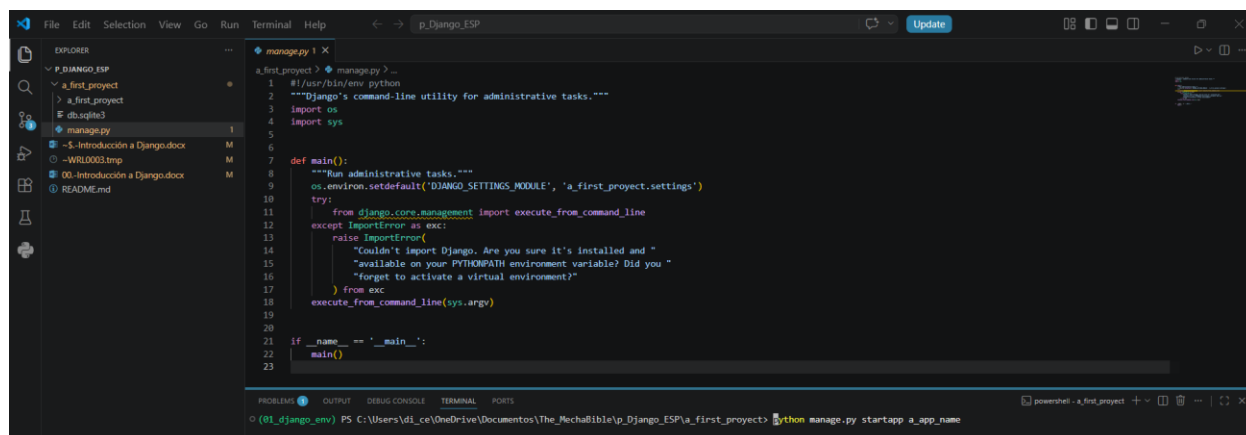
La arquitectura MVT (Model View Template) es la utilizada dentro del framework de Django, esa está pensada para

- **Model (M):** Esta es la capa de datos de la arquitectura, que indica como se guardan los datos, donde se procesan, que tipo de datos son, donde va la lógica de negocio, etc.
 - →: El **modelo** entrega estructuras de datos del backend hacia la vista.
 - ←: El **modelo** recibe una petición de la **vista** y entrega hacia el **template** el dato que está buscando.
- **View (V):** Esta se refiere a un conector entre el modelo que tiene acceso a los datos y el template (UI) que los está pidiendo para saber si los puede proporcionar o no, esta parte es la encargada de controlar el flujo de peticiones SQL de la interfaz gráfica hacia la base de datos y respuestas del backend hacia el frontend.
 - →: La **vista** entrega al **template** un array de datos recibidos del modelo en un tipo de estructura llamada contexto, que es un diccionario de Python con información cambiante de la base de datos, para que se pueda mostrar al usuario en la UI (user interface).
 - ←: La **vista** recibe una petición (id, request y query) del template del **template** y se conectará al **modelo** para encontrar el dato.
- **Template (T):** Se refiere a la interfaz gráfica de la aplicación, ósea donde se visualizan los datos del modelo.
 - →: El **template** recibe la estructura de la **vista** y la muestra al usuario.
 - ←: El **template** entrega una petición a la **vista**, para buscar un dato en el modelo, indicando el usuario (id asignado al user en la base de datos), el request (método HTTP hecho a la API) y la query (petición SQL hacia la base de datos).



Nota: Aunque se puede hacer una conexión directa entre el modelo y el template, no es de buenas prácticas, ya que la vista realiza funcionalidades clave en la arquitectura como comprobar los permisos del usuario (autenticación), roles, etc.

- **python manage.py startapp a_app_name**: El método **startapp** sirve para crear una aplicación dentro de nuestro proyecto previamente creado con el comando **django-admin startproject a_project_name**, al ejecutar este comando se creará una carpeta con el nombre **a_app_name**, que contendrá varios archivos, incluidos los scripts de **models.py** y **views.py** de la arquitectura MVT, para esto debemos recordar que ya debemos haber encendido nuestro entorno virtual con el comando **& \$HOME/.envs/01_django_env/Scripts/Activate.ps1** dentro de la carpeta del proyecto **a_project_name**.

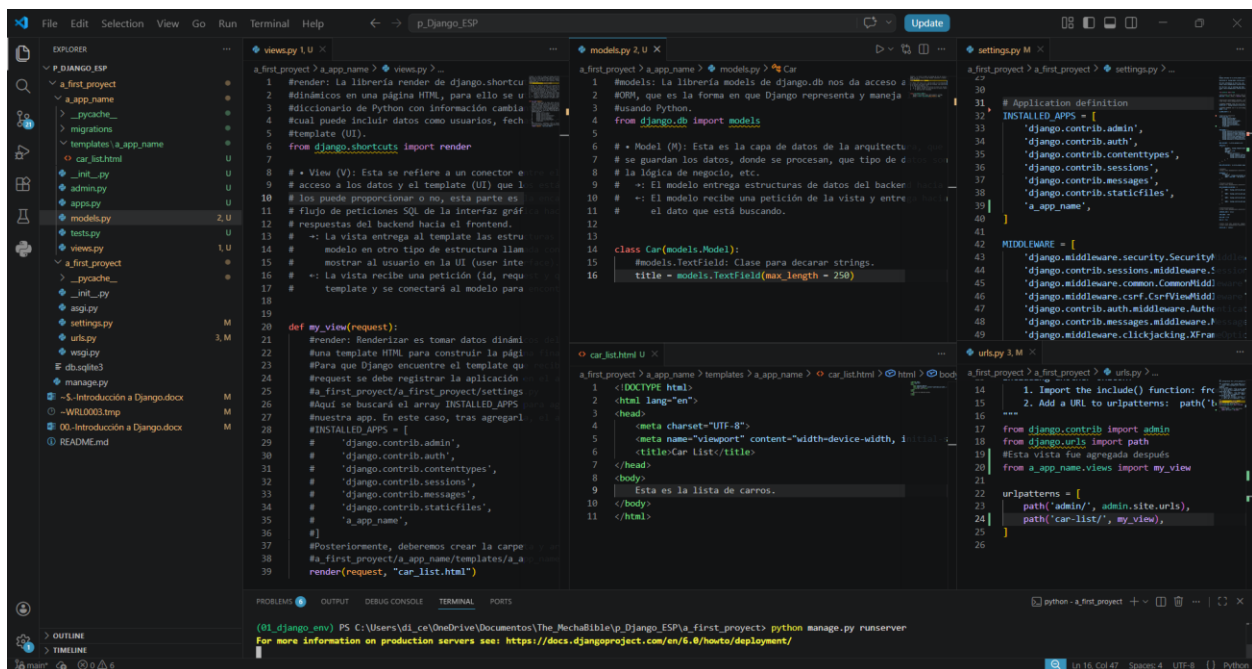


- **models.py**: Una vez que el proyecto y la app han sido creadas, dentro del script **models.py** deberemos crear las clases que reflejen las tablas de las bases de datos que se crearán y manejarán en la operación, esto se hace a través de la librería **models** de **django.db** a través de **ORM (Object-Relational Mapping)**, que es la forma en que Django representa y maneja la base de datos usando Python.
- **views.py**: En el script **views.py** se crearán los renders, que cargan datos dinámicos en una página HTML, para ello se utiliza un contexto, que es un diccionario de Python con información cambiante de la base de datos, la cual puede incluir datos como usuarios, fechas, etc. y los inserta en el **template (UI)**.

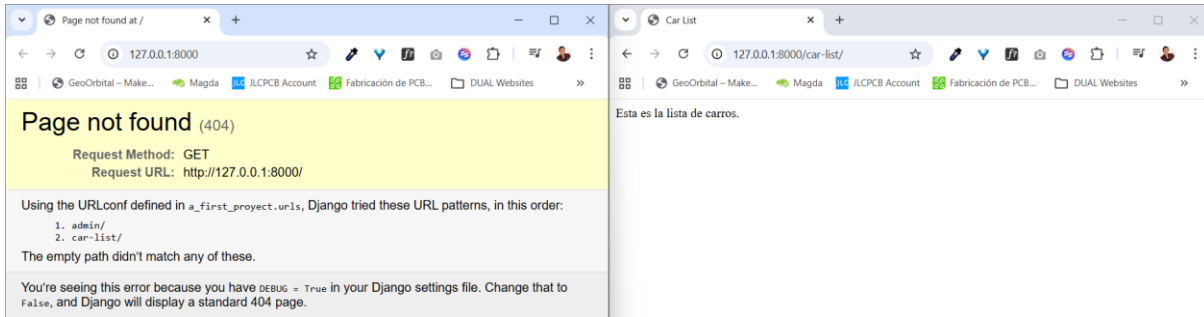
- **settings.py:** Ya habiendo creado una función dentro de **views.py** que reciba un request y lo asigne a un render, este se deberá asignar a un template, para ello se debe registrar la aplicación en el archivo **a_first_proyect/a_first_proyect/settings.py**, aquí se buscará el array **INSTALLED_APPS** para agregar el nombre de nuestra app. En este caso, tras agregarla, el array quedará así:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'a_app_name',
]
```

- Posteriormente, deberemos crear la carpeta y archivo del template: **a_first_proyect/a_app_name/templates/a_app_name/template_1.html**, luego colocar un return en la función del render y actualizar el path de este archivo, solamente añadiendo lo que diga después de **templates/**.
- **urls.py:** Finalmente, para que podamos acceder a esa vista desde el proyecto, se debe agregar un nuevo path que haga referencia a la función que declaró el render de nuestra app en el script **views.py** y se ejecutará el comando **python manage.py runserver** para verla en el navegador.



La página principal mostrará un error 404 ya que no hemos definido la página de inicio, pero si en el localhost colocamos el endpoint del path que acabamos de crear en el script de **urls.py**, aparecerá el **template** que acabamos de crear.

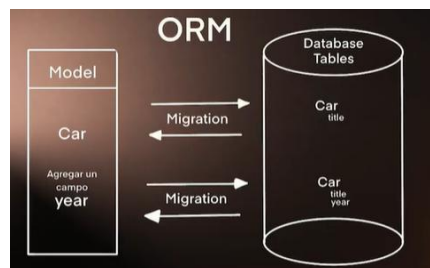


Model: ORM (Object-Relational Mapping) y Migraciones

La parte del Modelo en la arquitectura MVT se refiere a la conexión del código con la base de datos, en Django esto se maneja a través de un concepto llamado ORM (Object-Relational Mapping), que nos permite crear clases de Python que se relacionarán con tablas de la base de datos que elijamos, para no tener que escribir secuencias SQL y solo trabajar con el lenguaje Python, donde:

Clase Script = Tabla DB.

Para poder crear el contenido de una clase de Python dentro de una tabla SQL Django nos proporciona una herramienta llamada migraciones, estas nos permiten hacer cambios en la base de datos, ya sea para realizar desde Python un cambio en la database, o desde la base de datos, un cambio en la clase de Python, para que ambas estén sincronizadas.



Cuando todavía no se han creado las tablas de nuestras clases del archivo model.py, al correr el archivo manage.py de Django, aparecerá el siguiente error, el cual no desaparecerá hasta que se creen las migraciones de todas nuestras clases declaradas en el archivo de **models.py**, que representan las tablas de nuestra database:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\di_ce\OneDrive\Documentos\The_MechaBible\p_Django_ESP> & $HOME/.envs\01_django_env\Scripts/Activate.ps1
(01_django_env) PS C:\Users\di_ce\OneDrive\Documentos\The_MechaBible\p_Django_ESP> cd .\a_first_project\
(01_django_env) PS C:\Users\di_ce\OneDrive\Documentos\The_MechaBible\p_Django_ESP\> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
May 14, 2026 - 16:13:41
Django version 6.0.4, using settings 'a_first_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

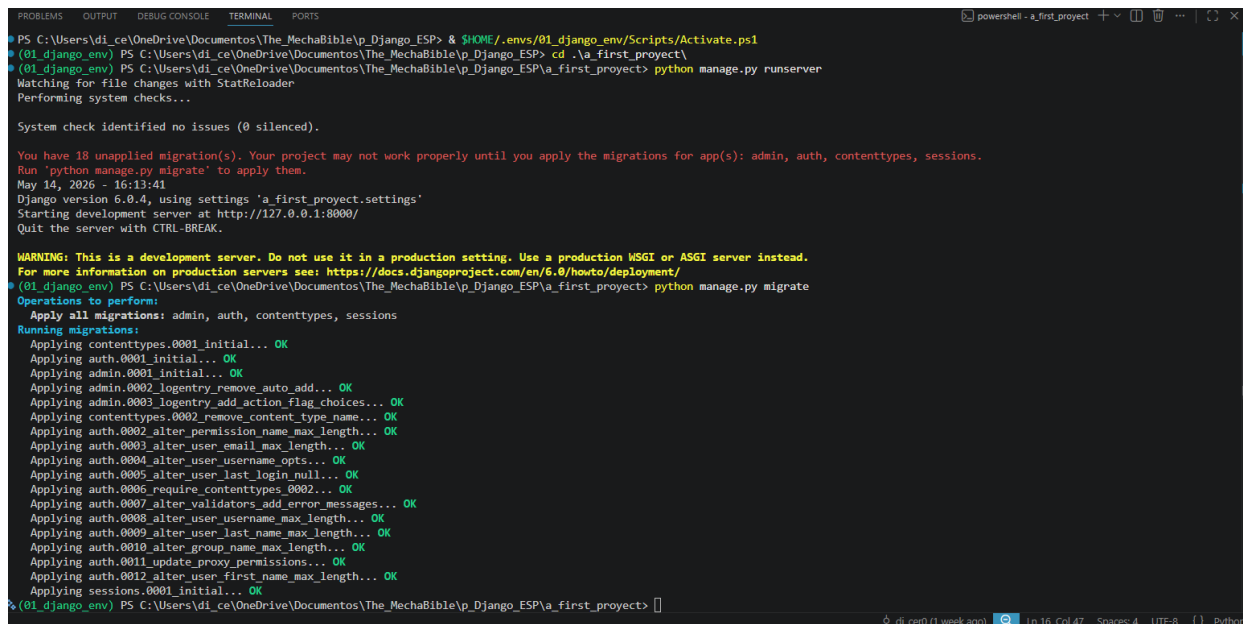
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/

```

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.

Para desde cero prender nuestro entorno virtual, correr el servidor de nuestro proyecto y ejecutar una migración se deben correr estos comandos en el siguiente orden:

- Activar el entorno virtual: **& \$HOME/.envs/01_django_env/Scripts/Activate.ps1**
- Luego nos debemos introducir a la carpeta de nuestro proyecto: **cd .\a_first_project**
- Posteriormente activar el servidor del proyecto: comando **python manage.py runserver**
- Y finalmente se corre el método **migrate** del comando **manage.py** para ejecutar una migración de las clases y funcionalidades incluidas por defecto en Django: **python manage.py migrate**



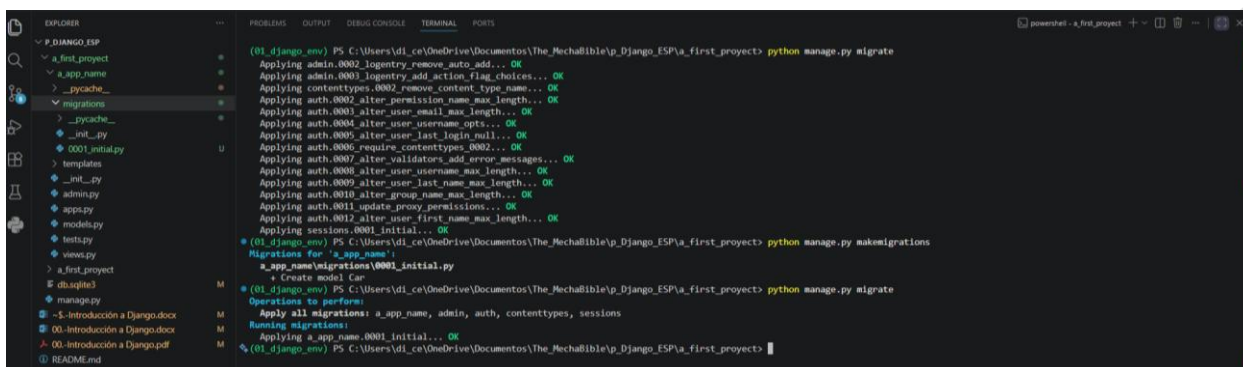
```
PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> & $HOME/.envs/01_django_env/Scripts/Activate.ps1
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> cd .\a_first_project\
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
May 14, 2026 - 16:13:41
Django version 6.0.4, using settings 'a_first_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP>
```

- Ya habiendo ejecutado las migraciones de Django, si queremos crear nuestras propias tablas, ósea las clases contenidas en el archivo **models.py**, debemos utilizar el siguiente comando, el cual creará dentro de la carpeta **a_app_name/migratons/** los archivos de migraciones: **python manage.py makemigrations**
- Los archivos de migración **.sql** o **.py** representan los cambios hechos a la base de datos, el primero que creará se llamará **a_app_name/migratons/0001_initial.py** y debemos saber que, una vez creado el archivo de la nueva migración, se debe ejecutar de nuevo el comando **python manage.py migrate** para que este se aplique a la base de datos.



```
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python manage.py migrate
Applying admin.0002_logentry_remove_auto_add... OK
Applying admin.0003_logentry_add_action_flag_choices... OK
Applying contenttypes.0002_remove_content_type_name... OK
Applying auth.0002_alter_permission_name_max_length... OK
Applying auth.0003_alter_user_email_max_length... OK
Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK
Applying auth.0006_require_contenttypes_0002... OK
Applying auth.0007_alter_validators_add_error_messages... OK
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying sessions.0001_initial... OK
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python manage.py makemigrations
Migrations for 'a_app_name':
  a_app_name/migrations/0001_initial.py
+ Create model Car
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> python manage.py migrate
Operations to perform:
  Apply all migrations: a_app_name, admin, auth, contenttypes, sessions
Running migrations:
  Applying a_app_name.0001_initial... OK
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP>
```

Base de Datos SQLite y su Modificación en Settings.py

La base de datos que se usa por defecto al crear un proyecto de Django es SQLite y se incluye en los archivos del proyecto, pero estas no se utilizan en producción ya que son de baja capacidad y concurrencia, ósea que no puede gestionar múltiples tareas o procesos independientes simultáneamente, por lo que cuando se quiera utilizar Django en producción, se debe utilizar otro motor de bases de datos como PostgreSQL, MongoDB, etc.

- Aunque no se puede utilizar la base de datos SQLite en producción, por fines didácticos o de diseño del sistema, se utiliza el siguiente comando para poder observar las tablas y datos de este tipo de database, para que este funcione, ya debemos haber instalado previamente SQLite:
python manage.py dbshell

Cabe mencionar, que cuando queramos modificar la base de datos utilizada en el proyecto, debemos modificar el archivo **settings.py**, el cual no solo sirve para conectar nuestras vistas del archivo **views.py** a un template a través del array **INSTALLED_APPS**, sino que de igual forma puede:

- **SECRET_KEY**: Contener una llave de encriptación.
- **DEBUG**: Booleano True o False que nos facilita debuggear la aplicación Django.
- **ALLOWED_HOST**: Array que incluye los dominios con los que se puede acceder a nuestra aplicación una vez que haya sido deployada (subida a la nube).
- **INSTALLED_APPS**: Son las aplicaciones que vienen preinstaladas de Django y además se puede agregar la nuestra para que las vistas de nuestro archivo **views.py** se puedan conectar a una UI de nuestra carpeta **templates**.
- **TEMPLATES**: Es un array que nos permite configurar la visualización de los componentes de la interfaz gráfica del proyecto (UI) dependiendo de su acceso por seguridad, entre otras cosas.
- **DATABASES**: Este array contiene una o varias bases de datos que se vayan a utilizar, indicando su engine (SQLite, PostgreSQL, MongoDB) y nombre.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

- **python manage.py dbshell**: Una vez ejecutado el comando para visualizar la base de datos que tengamos conectada al array de **DATABASES**, debemos ejecutar el comando **.tables** dentro del Shell de SQL para observar todas las tablas que existan en nuestra database, el nombre que le asignará a cada tabla que se cree a través del script **models.py**, será **a_app_name_nombreClase**, además con el comando **.schema a_app_name_nombreClase**, podremos ver todas las columnas de la tabla, si queremos ver sus datos deberemos ejecutar el query **SELECT * FROM a_app_name_nombreClase**;

```
PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> & $HOME/.envs/01_django_env/Scripts/Activate.ps1  
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP> cd .\a_first_project\  
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\first_project> python manage.py dbshell  
SQLite version 3.53.1 2026-05-05 10:34:17  
Enter ".help" for usage hints.  
sqlite> .tables  
a_app_name_car      auth_group_permissions  auth_user              auth_user_user_permissions  django_content_type  django_session  
auth_group          auth_permission         auth_user_groups       django_admin_log           django_migrations  
sqlite> .schema a_app_name_car  
CREATE TABLE "a_app_name_car" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" text NOT NULL);  
sqlite>
```

```

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py dbshell
Enter ".help" for usage hints.
sqlite> .tables
a_app_name_car      auth_group_permissions  auth_user            auth_user_user_permissions  django_content_type  django_session
auth_group          auth_permission        auth_user_groups     django_admin_log           django_migrations
sqlite> .schema a_app_name_car
CREATE TABLE "a_app_name_car" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" text NOT NULL, "year" text NULL);
sqlite> select * from a_app_name_car;

+----+-----+-----+
| id | title | year |
+----+-----+-----+
| 1  | Mazda | 2023 |
+----+-----+-----+

sqlite>

```

- **.quit**: Una vez que hayamos ejecutado el comando **python manage.py dbshell**, para salir de la consola de comandos de SQLite, debemos ejecutar el comando **.quit**.

Cada vez que se quiera hacer un cambio a una database existente con el archivo **models.py**, debemos ejecutar el comando **python manage.py makemigrations** y luego **python manage.py migrate** para que este se aplique a la base de datos, pero aquí vale la pena mencionar, que cada vez que se haga un cambio a una DB previamente creada, se le debe asignar un valor por defecto, para que este se asigne a las filas de datos anteriores a que esta nueva columna se agregara, ya sea NULL o un valor por específico por default y esto se muestra cuando se quiera agregar la migración.

```

a_first_project > a_app_name > models.py > Car
1 # Modelo: la librería models de django.db nos da acceso al sistema de modelos
2 # ORM, que es la forma en que Django representa y maneja la base de datos
3 # Usando Python.
4 from django.db import models
5
6 # = Model (M): Esta es la capa de datos de la arquitectura, que indica como
7 # se guardan los datos, donde se procesan, que tipo de datos son, donde va
8 # la lógica de negocio, etc.
9 # => El modelo entrega estructuras de datos del backend hacia la vista.
10 # => El modelo recibe una petición de la vista y entrega hacia el template
11 # el dato que está buscando.
12
13
14 class Car(models.Model):
15     #Cada uno de los atributos que se agreguen a la clase, representarán una
16     #columna en la tabla de la base de datos.
17     #models.TextField: Clase para declarar strings.
18     title = models.TextField(max_length=250)
19     year = models.TextField(max_length=4)

```

```

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py dbshell
SQLite version 3.33.1 2020-05-05 10:34:17
Enter ".help" for usage hints.
sqlite> .quit

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py makemigrations
It is impossible to add a non-nullable field "year" to car without specifying a default. This is because the database needs something to populate existing rows.
Please select a fix:
1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
2) Quit and manually define a default value in models.py.
Select an option:

```

Al crear la nueva migración, se creará otro archivo dentro de la carpeta **migrations**, que indicará todos los cambios hechos a la tabla en la DB.

```

a_first_project > a_app_name > migrations > 0002_car_year.py > Migration
1 # Generated by Django 6.0.4 on 2020-05-14 16:22
2 from django.db import migrations, models
3
4 class Migration(migrations.Migration):
5     dependencies = [
6         ('a_app_name', '0001_initial'),
7     ]
8     operations = [
9         migrations.AddField(
10             model_name='car',
11             name='year',
12             field=models.TextField(max_length=4, null=True),
13         ),
14     ]

```

```

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py dbshell
SQLite version 3.33.1 2020-05-05 10:34:17
Enter ".help" for usage hints.
sqlite> .quit

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py makemigrations
Migrations for 'a_app_name':
  a_app_name/migrations/0002_car_year.py
  Add field year to car

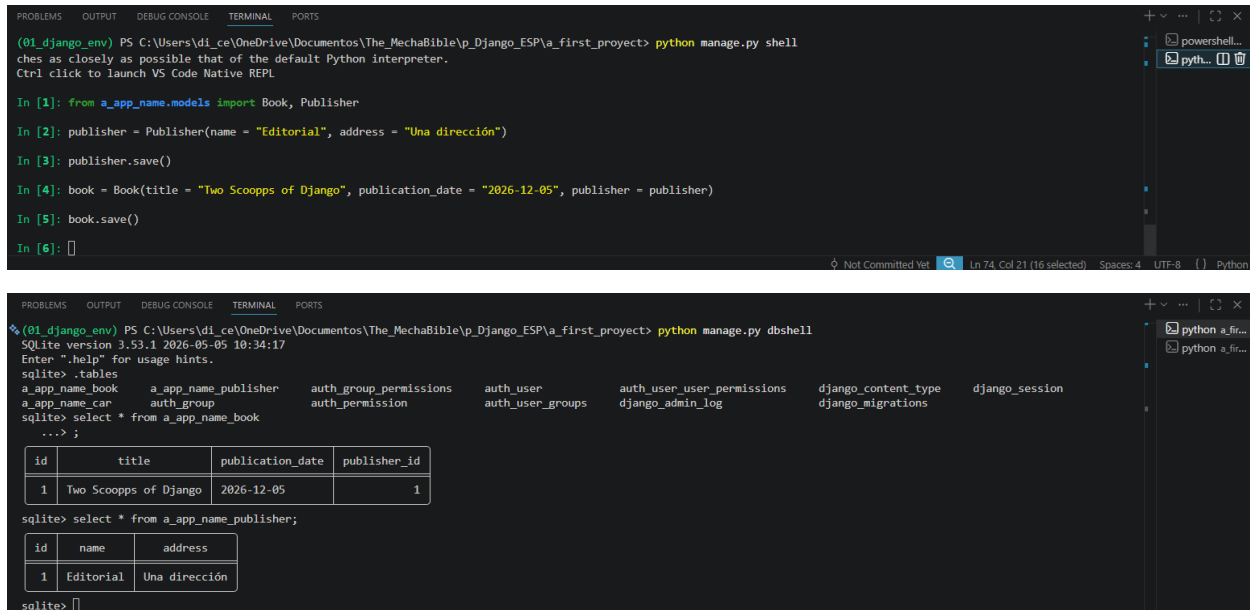
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py migrate
Operations to perform:
  Apply all migrations: a_app_name, admin, auth, contenttypes, sessions
Running migrations:
  Applying a_app_name.0002_car_year... OK

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project>

```

Todos los tipos de campos (tipos de datos) que se puede agregar a una database a través de Django se encuentran documentados en este enlace, como lo pueden ser TextField para texto, Choices para Arrays y JSONs, DateField para fechas, etc: <https://docs.djangoproject.com/en/5.0/ref/models/fields/>

Además, para que podamos escribir fácilmente código Python en consola, permitiéndonos añadir, eliminar y modificar datos en la DB a través de objetos, debemos instalar la librería iPython con el comando: **pip install ipython** y luego abrir la consola con **python manage.py shell**.



```
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_proyect> python manage.py shell
ches as closely as possible that of the default Python interpreter.
Ctrl click to launch VS Code Native REPL

In [1]: from a_app_name.models import Book, Publisher

In [2]: publisher = Publisher(name = "Editorial", address = "Una dirección")

In [3]: publisher.save()

In [4]: book = Book(title = "Two Scoopps of Django", publication_date = "2026-12-05", publisher = publisher)

In [5]: book.save()

In [6]: []
```

```
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_proyect> python manage.py dbshell
SQLite version 3.53.1 2026-05-05 10:34:17
Enter ".help" for usage hints.
sqlite> .tables
a_app_name_book      a_app_name_publisher  auth_group_permissions  auth_user      auth_user_user_permissions  django_content_type  django_session
a_app_name_car       auth_group            auth_permission         auth_user_groups  django_admin_log           django_migrations
sqlite> select * from a_app_name_book
***> ;


| id | title                 | publication_date | publisher_id |
|----|-----------------------|------------------|--------------|
| 1  | Two Scoopps of Django | 2026-12-05       | 1            |


sqlite> select * from a_app_name_publisher;


| id | name      | address       |
|----|-----------|---------------|
| 1  | Editorial | Una dirección |


sqlite> []
```

- **python manage.py shell**: Para poder modificar las tablas en la base de datos a través de Python, se utilizará el método shell, que es una terminal de Python que tiene nuestro proyecto de Django cargado automáticamente como si el proyecto estuviera corriendo, para ello utilizaremos código normal de POO (programación orientada a objetos) para introducir y visualizar los datos de nuestra database.
 - **__str__(self)**: El concepto de String en los modelos es un método cuya función es definir cómo se representa un objeto como texto cuando se imprime o se muestra en consola.

Tipos de Relaciones entre Tablas: Uno a Muchos, Muchos a Muchos y Uno a Uno

La idea principal de las relaciones es representar conexiones reales del mundo entre objetos sin duplicar información. Por ejemplo, en vez de guardar el nombre completo de una editorial en cada libro, se guarda únicamente el id de la editorial. Así, múltiples libros pueden apuntar a la misma editorial.

- **Llave foránea (Foreign Key)**: Es simplemente una columna que apunta al id de otra tabla para crear una relación entre ambas, teniendo en cuenta que la **Foreign Key SIEMPRE** va en la tabla que pertenece o depende de la otra.
 - La forma de diseñar las bases de datos en cuestión de sus relaciones es resolver la pregunta de, **¿Cuántos de A pueden relacionarse con B?** A esto se le llama **cardinalidad**.

- **Un A tiene muchos B - One-to-Many:** Es una relación donde un registro (fila) de una tabla (clase) puede estar relacionado con muchos registros (muchas filas de datos) de otra tabla. Se utiliza cuando existe una jerarquía o dependencia clara, como una editorial con muchos libros o una empresa con muchos empleados.
- **Un A tiene un solo B - One-to-One:** Es una relación donde un registro solo puede relacionarse con un único registro de la otra tabla, y viceversa. Se utiliza normalmente para separar información opcional, sensible o extendida de una entidad principal, como un usuario y su perfil o una persona y su pasaporte.
- **Muchos A tienen muchos B - Many-to-Many:** Es una relación donde múltiples registros de una tabla pueden relacionarse con múltiples registros de otra tabla al mismo tiempo. Como una sola Foreign Key no se puede representar esta conexión, se crea una tabla intermedia que almacena las relaciones entre ambas entidades. Se utiliza en escenarios como libros con varios autores, estudiantes inscritos en varios cursos o productos pertenecientes a múltiples categorías, en Python esta tabla nueva se crea al utilizar la foreign key de tipo `models.ManyToManyField`.

```

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py shell
17 objects imported automatically (use -v 2 for details).

Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.13.0 -- An enhanced Interactive Python. Type '?' for help.
Tip: We can't show you all tips on Windows as sometimes Unicode characters crash the Windows console, please help us debug it.
Ctrl click to launch VS Code Native REPL

In [1]: from a_app_name.models import Book, Publisher, Author, Profile

In [2]: author = Author.objects.first()

In [3]: profile = Profile(website = "https://example.com", biography = "This is my bio")

In [4]: profile = Profile(website = "https://example.com", biography = "This is my bio", author = author)

In [5]: profile.save()

In [6]:

```

```

(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py dbshell
SQLite version 3.53.1 2026-05-05 10:34:17
Enter ".help" for usage hints.
sqlite> tables
a_app_name_author  a_app_name_book_authors  a_app_name_profile  auth_group  auth_permission  auth_user_groups  django_admin_log  django_migrations
a_app_name_book  a_app_name_car  a_app_name_publisher  auth_group_permissions  auth_user  auth_user_user_permissions  django_content_type  django_session
sqlite> select * from a_app_name_profile;

```

id	website	biography	author_id
1	https://example.com	This is my bio	1

```

sqlite> select * from a_app_name_author;

```

id	name	birth_date
1	Audrey	1995-04-06
2	Pydanny	1997-10-25

```

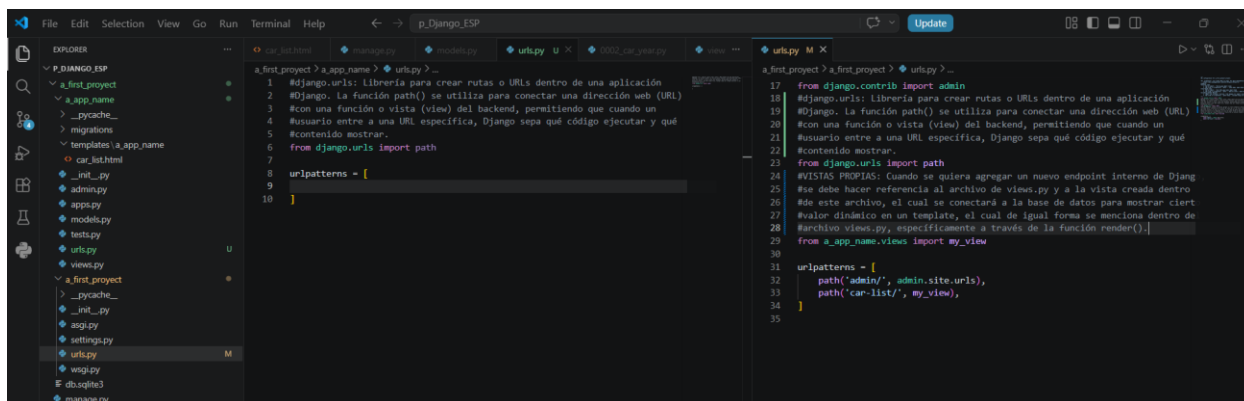
sqlite>

```

URLs.py: Archivo de Configuración de Endpoints

Ya existe un archivo predeterminado dentro de la carpeta `a_first_project/a_first_project/urls.py`, donde se utiliza la librería `Django.urls`, la cual sirve para añadir los endpoints que serán desplegados hacia el usuario, ayudando así que los templates (interfaz de usuario o UI) y las vistas del archivo `a_first_project/a_app_name/views.py` puedan conectarse a las bases de datos y mostrarlos cuando se hagan requests a los diferentes elementos de las páginas, renderizando así su información, pero hay una recomendación de buenas prácticas que indican que las URLs deben estar agrupadas por aplicación, para

así tener todos sus endpoints centralizados, para ello se creará un nuevo archivo desde cero en el directorio de **a_first_project/a_app_name/urls.py**, teniendo solo el import de path, proveniente de django.urls y la lista urlpatterns vacía.

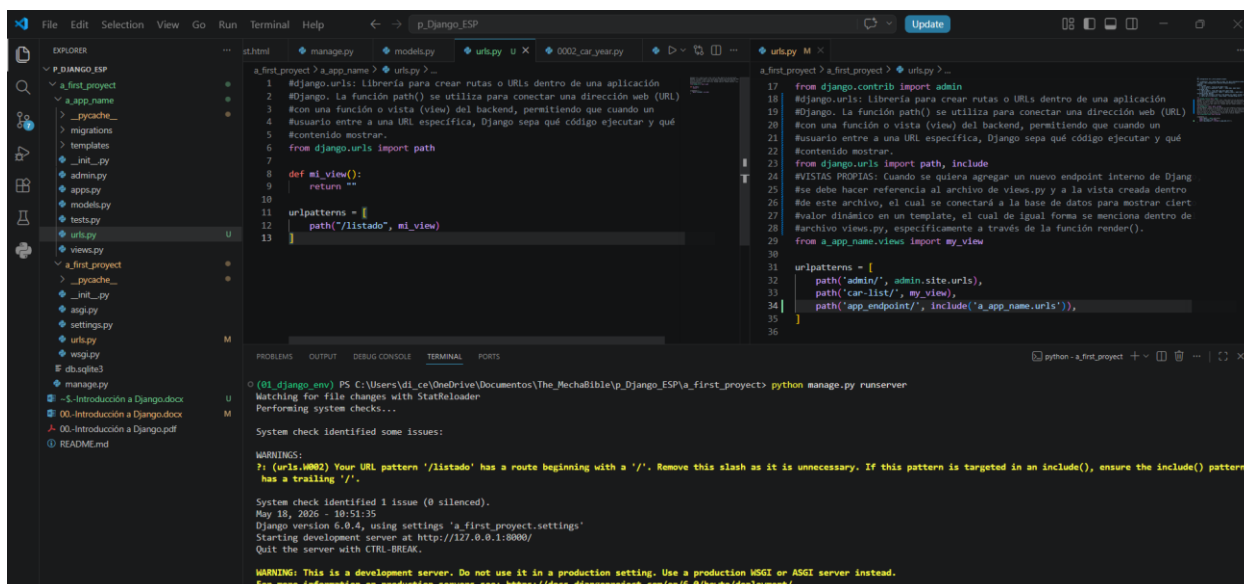


The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows the project structure with a new file `urls.py` created under `a_first_project/a_app_name/`. The Editor shows the content of `urls.py`, which includes imports for `django.urls` and `path`, and an empty `urlpatterns` list.

```
1 #django.urls: Librería para crear rutas o URLs dentro de una aplicación
2 #Django. La función path() se utiliza para conectar una dirección web (URL)
3 #con una función o vista (view) del backend, permitiendo que cuando un
4 #usuario entre a una URL específica, Django sepa qué código ejecutar y qué
5 #contenido mostrar.
6 from django.urls import path
7
8 urlpatterns = [
9
10 ]
```

Después, para que podamos incluir el archivo de URLs de nuestra aplicación **a_first_project/a_app_name/urls.py**, en el script de URLs de nuestro proyecto **a_first_project/a_first_project/urls.py**, lo que debemos hacer es importar la función `include()` de la librería `django.urls` y añadir el path del archivo `urls.py` de la carpeta **a_app_name**, con la sintaxis de módulo de Python, esta dicta que cada carpeta que tenga dentro de ella un archivo `__init__.py` se considera como un módulo y todos los scripts que contenga se considerarán como extensiones del módulo, teniendo la sintaxis de **'módulo.extensión'** y entonces como la carpeta de nuestra app cuenta con un archivo `init` y además tiene contenido el script `urls.py`, la sintaxis de módulo para importar ese archivo a través de la función `include` es: **`include('a_app_name.urls')`** y así es como se importan las URLs de la aplicación dentro del archivo de URLs global del proyecto.

Nuestra lista de endpoints disponibles aparecerá al ejecutar el comando **`python manage.py runserver`**, donde en un inicio no aparece nada debido a que no tenemos ningún template asociado al endpoint root (`/`), pero si aparece una lista de todas las endpoints disponibles y podemos ver que se ha añadido la de la aplicación.

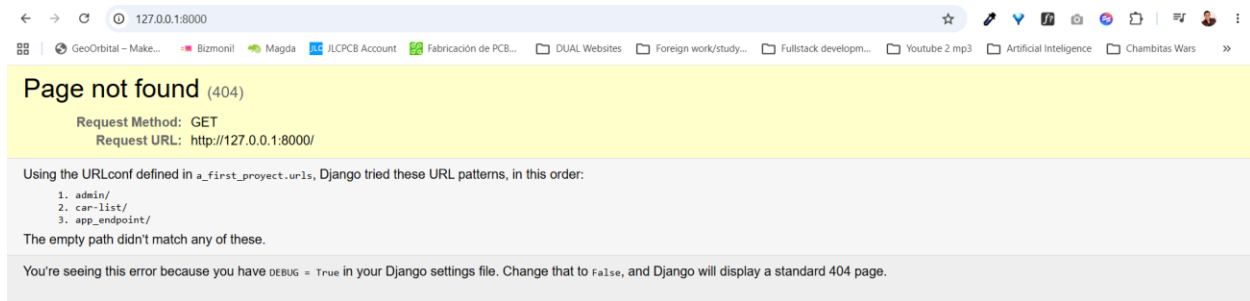


The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows the project structure with a new file `urls.py` created under `a_first_project/a_app_name/`. The Editor shows the content of `urls.py`, which includes imports for `django.urls` and `path`, and an empty `urlpatterns` list. The terminal at the bottom shows the output of the `python manage.py runserver` command, indicating that the server is running and displaying the Django version and settings.

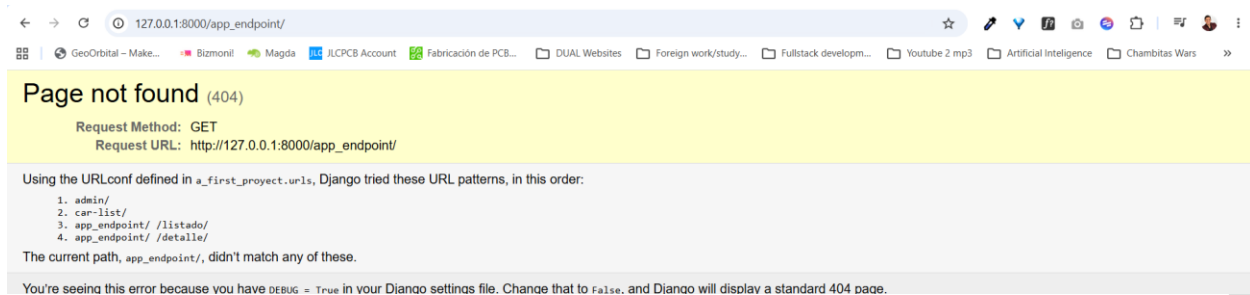
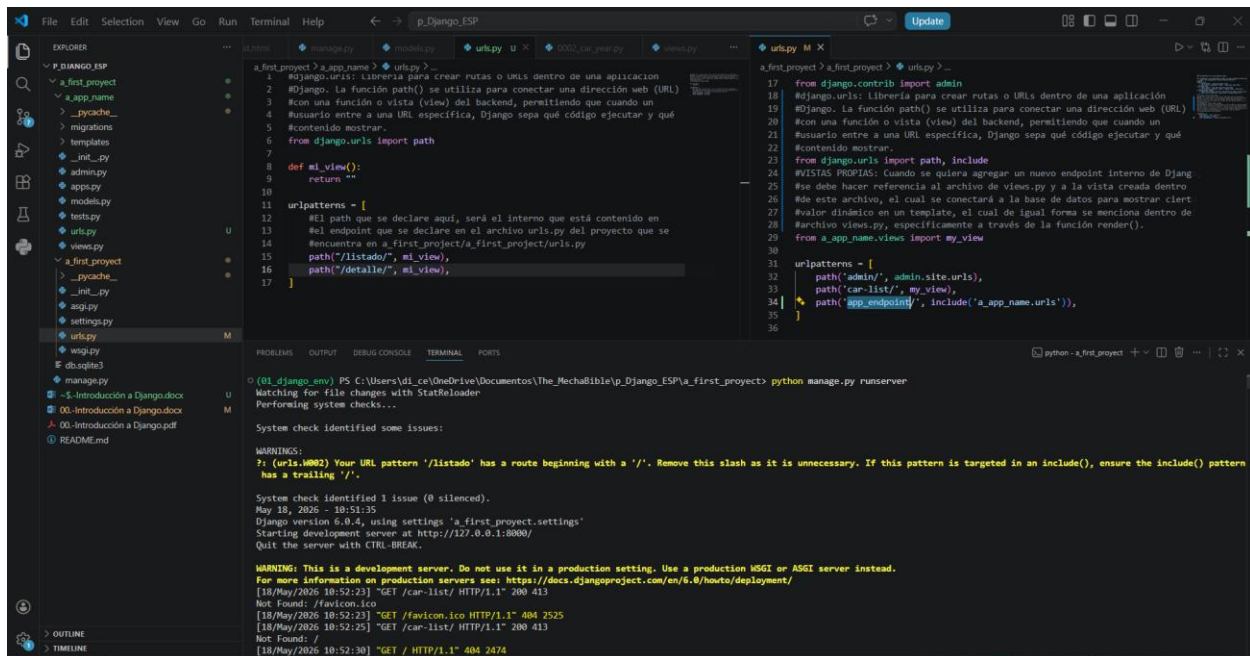
```
1 #django.urls: Librería para crear rutas o URLs dentro de una aplicación
2 #Django. La función path() se utiliza para conectar una dirección web (URL)
3 #con una función o vista (view) del backend, permitiendo que cuando un
4 #usuario entre a una URL específica, Django sepa qué código ejecutar y qué
5 #contenido mostrar.
6 from django.urls import path
7
8 urlpatterns = [
9
10 ]
```

```
17 from django.contrib import admin
18 #django.urls: Librería para crear rutas o URLs dentro de una aplicación
19 #Django. La función path() se utiliza para conectar una dirección web (URL)
20 #con una función o vista (view) del backend, permitiendo que cuando un
21 #usuario entre a una URL específica, Django sepa qué código ejecutar y qué
22 #contenido mostrar.
23 from django.urls import path
24 #VISTAS PROPIAS: Cuando se quiera agregar un nuevo endpoint interno de Django
25 #se debe hacer referencia al archivo de views.py y a la vista creada dentro
26 #de este archivo, el cual se conectará a la base de datos para mostrar cierta
27 #valor dinámico en un template, el cual de igual forma se menciona dentro de
28 #archivo views.py, específicamente a través de la función render().
29 from a_app_name.views import my_view
30
31 urlpatterns = [
32     path('admin/', admin.site.urls),
33     path('car-list/', my_view),
34     path('app_endpoint/', include('a_app_name.urls')),
35 ]
36
```

```
PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py runserver
Matching for file changes with StatReloader
Performing system checks...
System check identified some issues:
WARNINGS:
?: (urls.W002) Your URL pattern '/listado' has a route beginning with a '/'. Remove this slash as it is unnecessary. If this pattern is targeted in an include(), ensure the include() pattern has a trailing '/'.
System check identified 1 issue (0 silenced).
May 18, 2026 - 10:51:35
Django version 6.0.4, using settings 'a_first_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

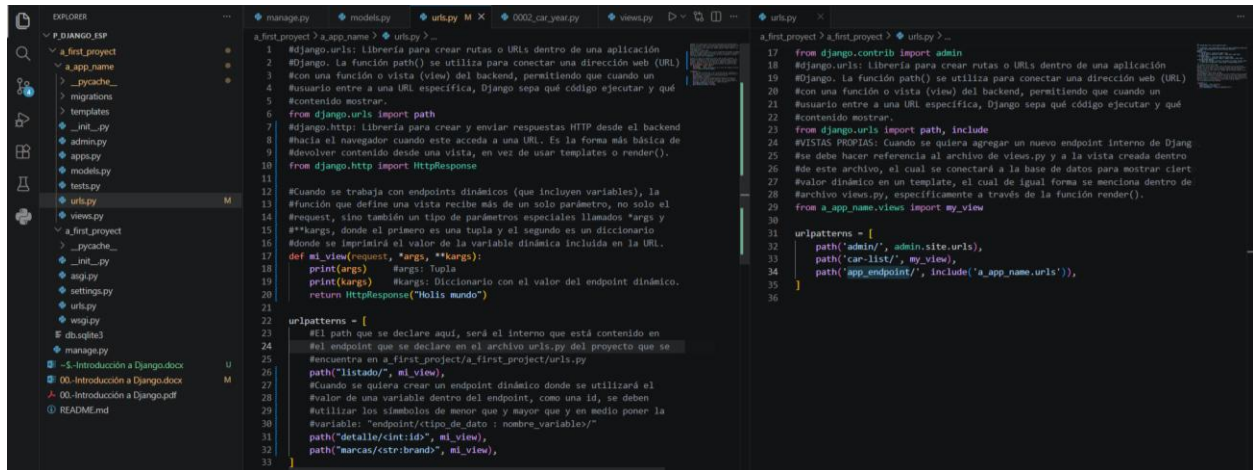



Los paths que se declaren en `a_first_proyect/a_app_name/urls.py`, serán los internos que estarán contenidos en el endpoint que se declare en el archivo `urls.py` del proyecto que se encuentra en `a_first_proyect/a_first_proyect/urls.py`, al que se puede acceder a través del navegador, donde serán visibles estos.

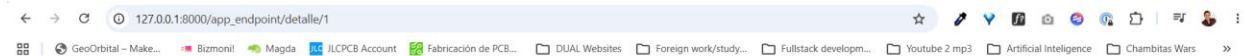


Vale la pena mencionar que de igual forma se pueden incluir valores variables dentro de nuestros endpoints cuando se declaren en los archivos `urls.py` a través de la sintaxis: `path("endpoint/<tipo_de_datos : nombre_variable>/", mi_view)`

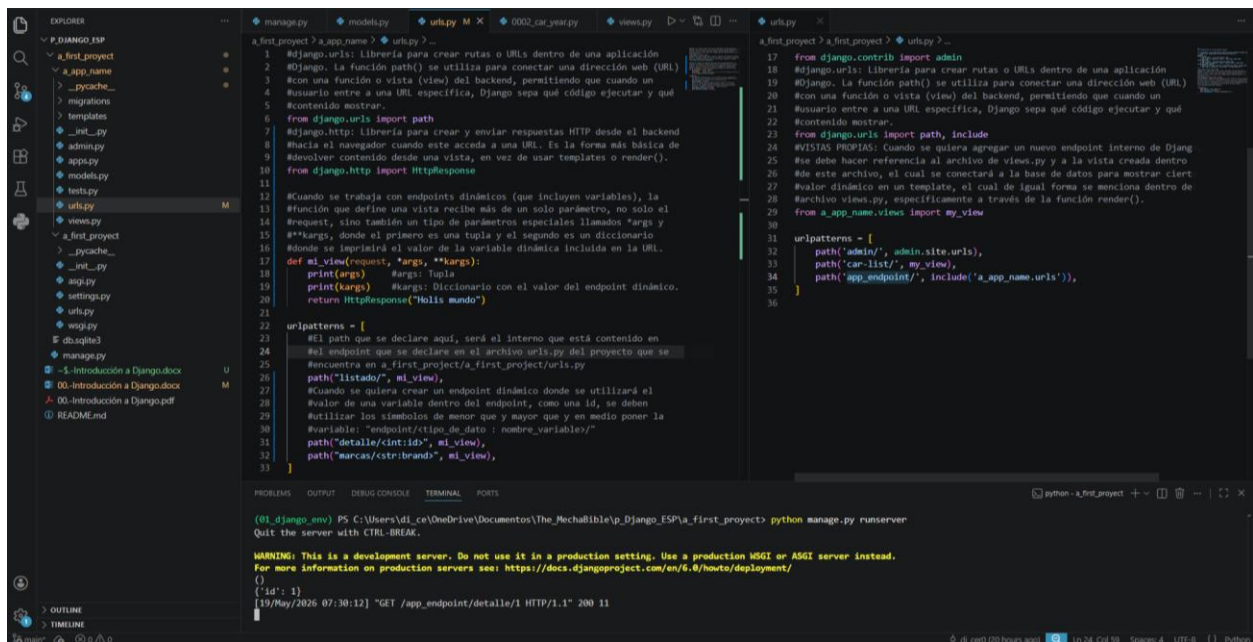
Pero las **funciones de tipo vista (mi_view)** que estas utilicen deben de no solo recibir el parámetro **request**, sino que también reciben ***args** y ***kwargs**, que corresponden a una tupla y un diccionario que muestran el valor que se está asignando a dichas variables de los endpoints a través de la URL del request.



```
1 #django.urls: Librería para crear rutas o URLs dentro de una aplicación
2 #django: La función path() se utiliza para conectar una dirección web (URL)
3 #con una función o vista (view) del backend, permitiendo que cuando un
4 #usuario entre a una URL específica, Django sepa qué código ejecutar y qué
5 #contenido mostrar.
6 from django.urls import path
7 #django.http: Librería para crear y enviar respuestas HTTP desde el backend
8 #función que define una vista recibe más de un solo parámetro, no solo el
9 #request, sino también un tipo de parámetros especiales llamados "args" y
10 #**kwargs, donde el primero es una tupla y el segundo es un diccionario
11 #donde se imprimirá el valor de la variable dinámica incluida en la URL.
12 #Cuando se trabaja con endpoints dinámicos (que incluyen variables), la
13 #función que define una vista recibe más de un solo parámetro, no solo el
14 #request, sino también un tipo de parámetros especiales llamados "args" y
15 #**kwargs, donde el primero es una tupla y el segundo es un diccionario
16 #donde se imprimirá el valor de la variable dinámica incluida en la URL.
17 def mi_view(request, *args, **kwargs):
18     print(args) #args: Tupla
19     print(kwargs) #kwargs: Diccionario con el valor del endpoint dinámico.
20     return HttpResponse("Holis mundo")
21
22 urlpatterns = [
23     #El path que se declare aquí, será el interno que está contenido en
24     #el endpoint que se declare en el archivo urls.py del proyecto que se
25     #encuentra en a_first_project/a_first_project/urls.py
26     path("listado/", mi_view),
27     #Cuando se quiera crear un endpoint dinámico donde se utilizará el
28     #valor de una variable dentro del endpoint, como una id, se deben
29     #utilizar los símbolos de menor que y mayor que y se medio poner la
30     #variable: "endpoint/tipo de dato : nombre_variable/"
31     path("detalle/<int:id>", mi_view),
32     path("marcas/<str:brand>", mi_view),
33 ]
```

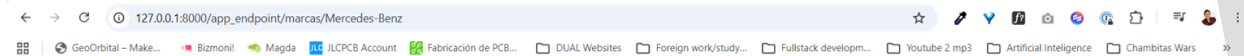


Holis mundo



```
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py runserver
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
()
{'id': 1}
[19/May/2026 07:30:12] "GET /app_endpoint/detalle/1 HTTP/1.1" 200 11
()
{'brand': 'Mercedes-Benz'}
[19/May/2026 07:32:12] "GET /app_endpoint/marcas/Mercedes-Benz HTTP/1.1" 200 11
```



Holis mundo



```
(01_django_env) PS C:\Users\di_ce\OneDrive\Documents\The_MechaBible\p_Django_ESP\p_a_first_project> python manage.py runserver
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
()
{'id': 1}
[19/May/2026 07:30:12] "GET /app_endpoint/detalle/1 HTTP/1.1" 200 11
()
{'brand': 'Mercedes-Benz'}
[19/May/2026 07:32:12] "GET /app_endpoint/marcas/Mercedes-Benz HTTP/1.1" 200 11
```


Django Templates: Variables, Filtros y Tags

- **Variables y Filtros:** Dentro de los templates que utiliza Django, se pueden declarar variables a través de doble llave `{{}}` y también se pueden indicar filtros a través de pipelines (`|`) a través de la siguiente sintaxis:

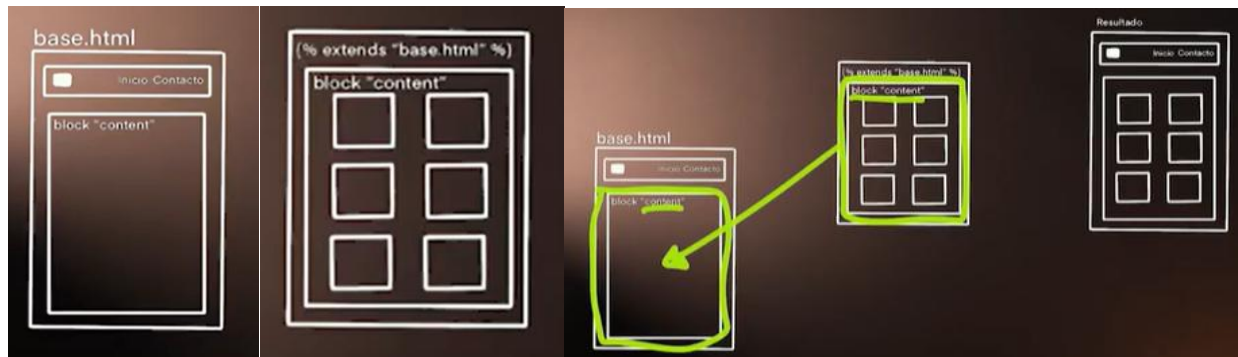
```
{{variable | filtro_1 | filtro_2}}
```

- **Tags:** Se denotan con el símbolo de porcentaje `%` y llaves simples `{}`, permitiéndonos añadir funcionalidad a nuestro código HTML, como lo pueden ser ciclos `for`, condicionales y urls. De igual forma se debe añadir un tag de cierre con el nombre `endif` o `endfor`.

```
{{%if, for o url%}}
```

```
{{%endif o endfor%}}
```

De igual forma, tenemos algo llamado `Base.html`, el cual se refiere a un archivo que nos permite tener contenido que es común dentro de nuestra aplicación, para que pueda ser reutilizado sin que se tenga que reescribir todo el código. En la parte inferior podemos observar un bloque, si luego se crea otro elemento del template que lo extienda, lo que se cree dentro de este podrá ser reutilizado en otras partes del código, teniendo así módulos repetibles con información dinámica, de forma parecida a como se hace con `React.js`

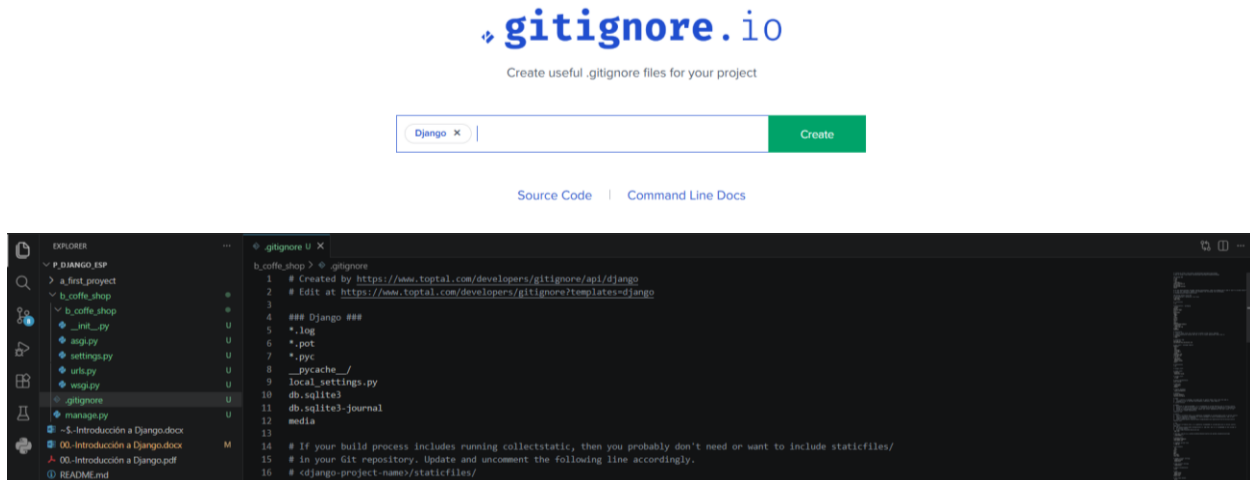


Ejemplo: Proyecto Coffee Shop

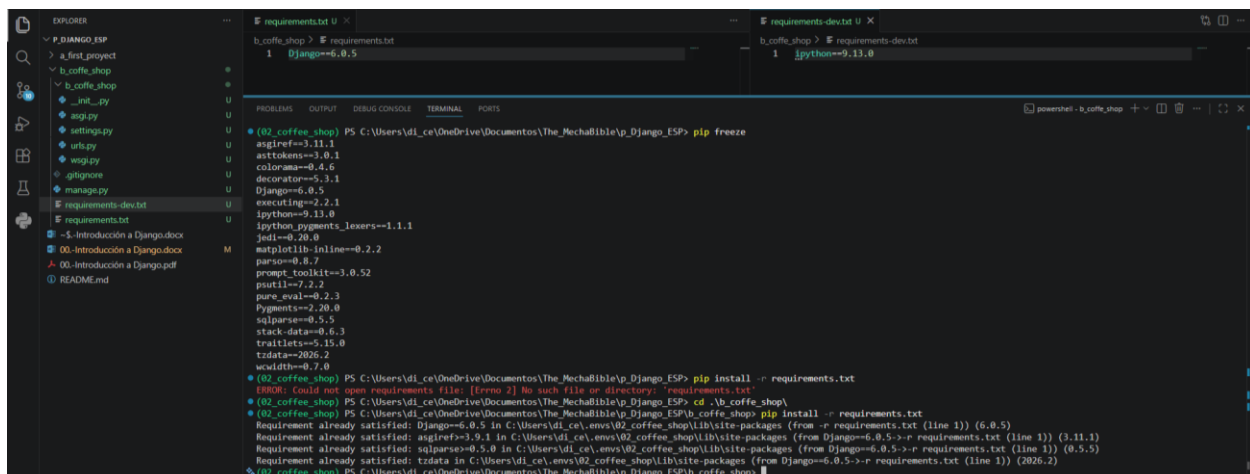
Recodemos ahora entonces el orden de ejecución que se debe seguir para crear un proyecto en Django:

1. Crear entorno virtual: **python -m venv \$HOME/.envs/nombre_entorno:**
 - a. `python -m venv $HOME/.envs/02_coffee_shop`
2. Activar entorno virtual: **& \$HOME/.envs/nombre_entorno/Scripts/Activate.ps1.**
 - a. `& $HOME/.envs/02_coffee_shop/Scripts/Activate.ps1`
 - i. Cuando se quiera desactivar, usar el comando: **deactivate**
3. Luego se debe instalar django en el entorno virtual: **pip install django.**
4. Crear el proyecto de Django: **django-admin startproject nombre_proyecto.**
 - a. `django-admin startproject b_coffe_shop`

5. Opcionalmente vale la pena añadir al repositorio un archivo `.gitignore`, el cual puede ser generado de forma automática en el siguiente link, al indicar la tecnología que se está utilizando y simplemente copiando y pegando su contenido al archivo:
- <https://www.toptal.com/developers/gitignore>



6. Después para el manejo de dependencias tenemos dos opciones, una es utilizar poetry y otra es declarar un archivo de `requirements.txt` donde se declararán de forma manual todas las versiones de las dependencias utilizadas en el proyecto, a continuación, describiremos ambas:
- requirements.txt:** Para realizar este método, se debe ejecutar el comando **pip freeze**, para luego copiar y pegar en el archivo de `requirements` su contenido.
 - Requirements-dev.txt:** Este archivo auxiliar de la misma forma se crea de forma manual y sirve para indicar las dependencias que queramos instalar, pero que no vayan a afectar producción, sino solamente el entorno de desarrollo, como lo es `iPython`, que nos sirve para poder observar de forma más estética el código Python que se mande a la base de datos para probar nuestros modelos, para ello lo que haremos es instalar la librería con el comando **pip install ipython** y luego volver a ejecutar **pip freeze** para ver su versión.
 - Para instalar las dependencias de dichos archivos se debe ejecutar el comando:
pip install -r requirements.txt



b.

Referencias

Platzi, Luis Martínez, “Curso de Django”, 2024 [Online], Available: <https://platzi.com/cursos/django/ques-django/>

